

Introduction to Theory of Safe Decision Making

Akhil S Anand

Department of Engineering Cybernetics
Faculty of Information Technology
NTNU

1st AID Scientific Workshop, Trondheim

Forewords & Disclaimer

Objectives:

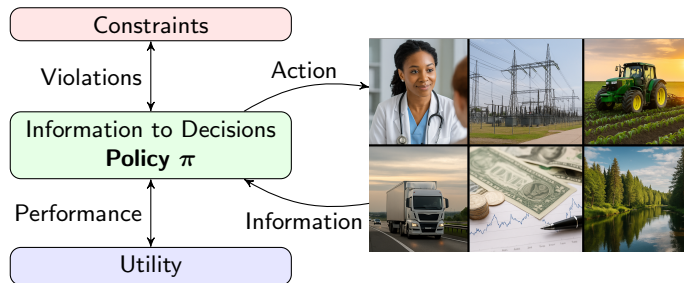
- Put in place some common concepts & language
- Identify some key points in safe decision making
- Connect to AID



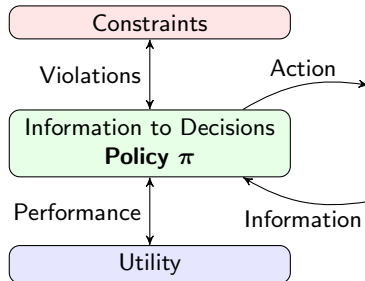
Outline

- 1 Some Basics of Safe Decision Making
- 2 Safe Decisions from Data & AI
- 3 Epistemic Uncertainty and Safe Decisions

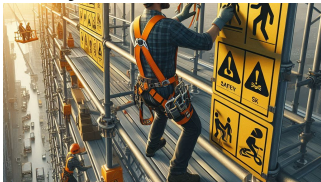
Formally Defining Safety?



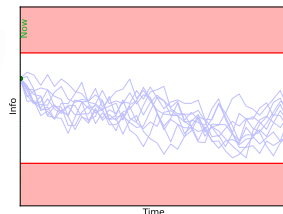
Formally Defining Safety?



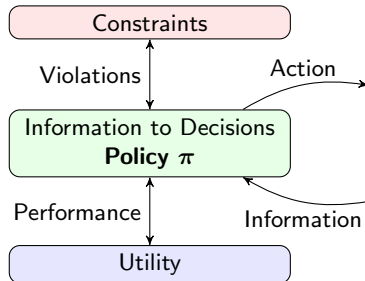
Safety in the real world



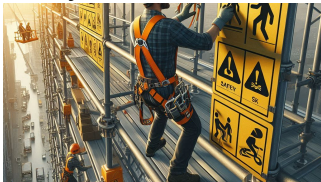
Safety in the math world



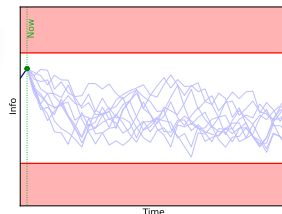
Formally Defining Safety?



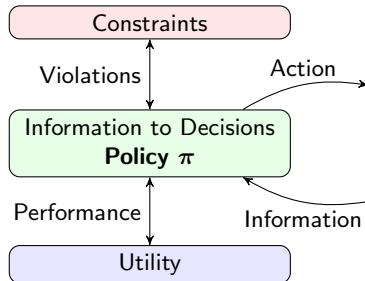
Safety in the real world



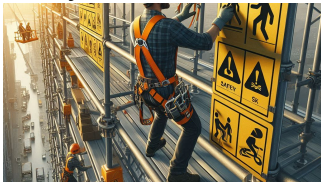
Safety in the math world



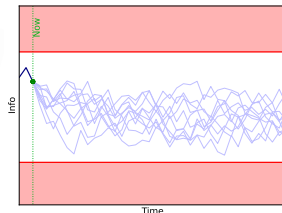
Formally Defining Safety?



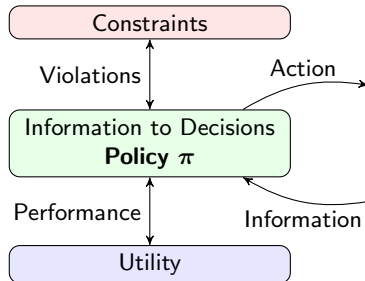
Safety in the real world



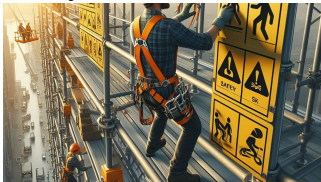
Safety in the math world



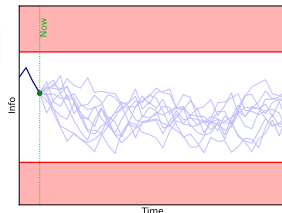
Formally Defining Safety?



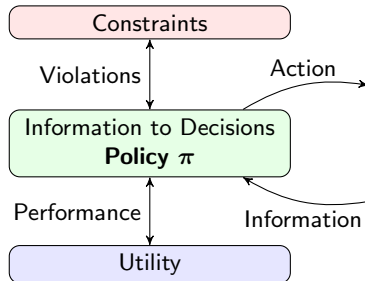
Safety in the real world



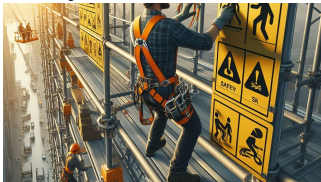
Safety in the math world



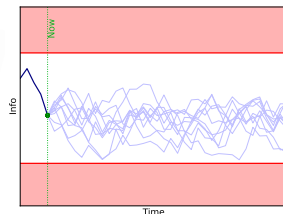
Formally Defining Safety?



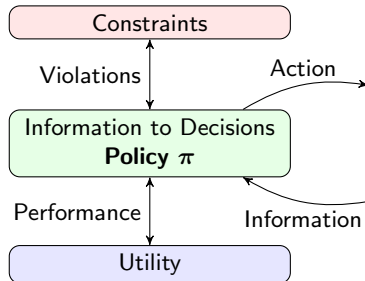
Safety in the real world



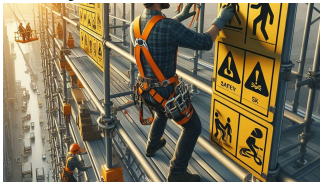
Safety in the math world



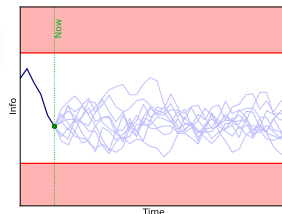
Formally Defining Safety?



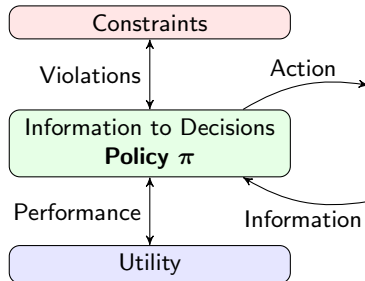
Safety in the real world



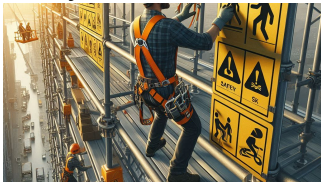
Safety in the math world



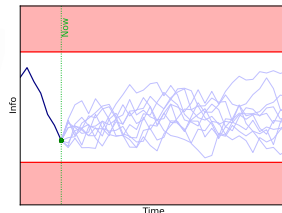
Formally Defining Safety?



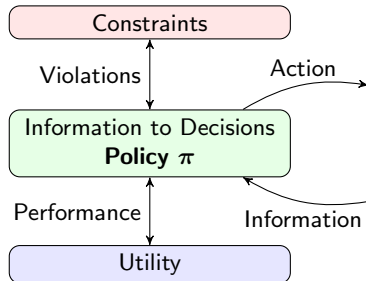
Safety in the real world



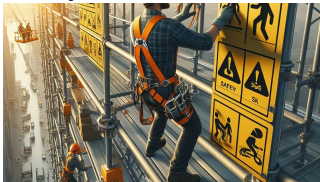
Safety in the math world



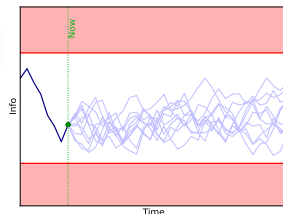
Formally Defining Safety?



Safety in the real world



Safety in the math world

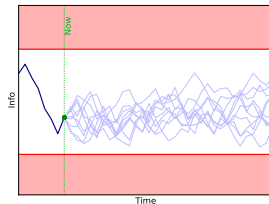


MDPs with Hard Constraints

In words

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Constraints ok at all time

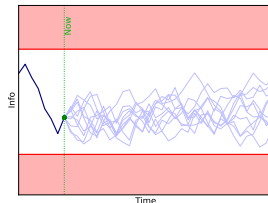


MDPs with Hard Constraints

In words

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Constraints ok at all time



Formally

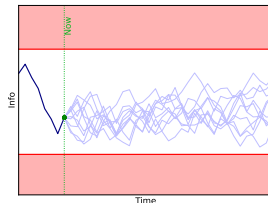
$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ \infty & \text{if Constraint not ok} \end{cases} \right) \right]$$

MDPs with Hard Constraints

In words

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Constraints ok at all time



Formally

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ \infty & \text{if Constraint not ok} \end{cases} \right) \right]$$

Build policy using either:

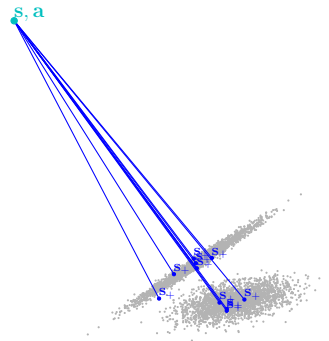
- Perfect model of the real world $\hat{P}[s_+|s, a] = P[s_+|s, a]$
- Model “pessimistic” about the uncertainties

... to evaluate “ $\mathbb{E}[\cdot]$ ”

Pessimistic Models for Decision Making

- Model must “contain” the uncertainty

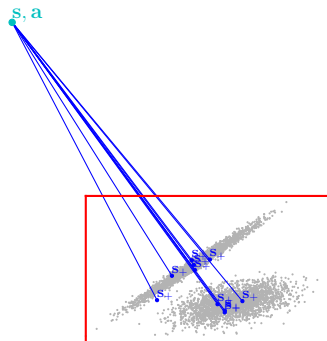
Distribution of one-step forward



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons

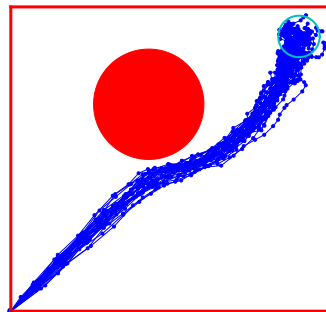
Distribution of one-step forward with “container”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

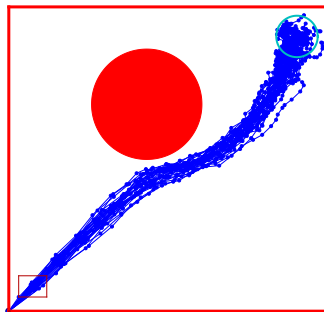
Trajectories



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

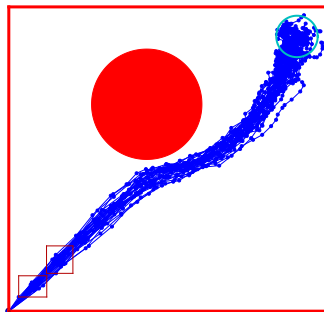
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

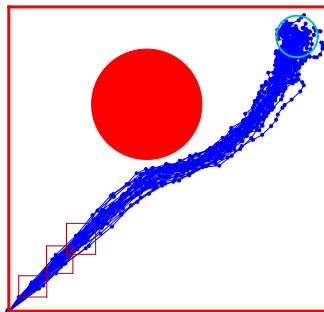
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

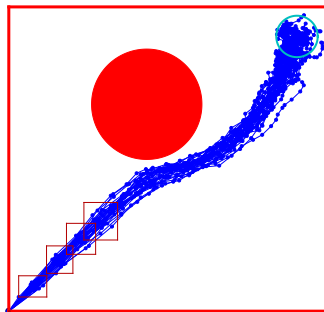
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

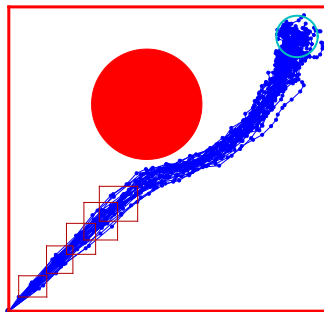
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

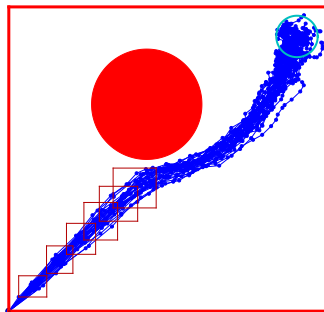
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

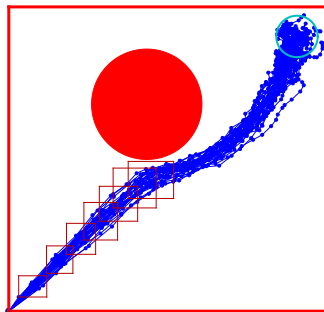
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

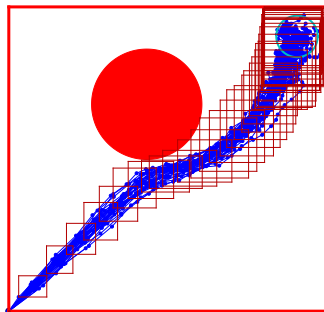
Trajectories with “containers”



Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

Trajectories with “containers”



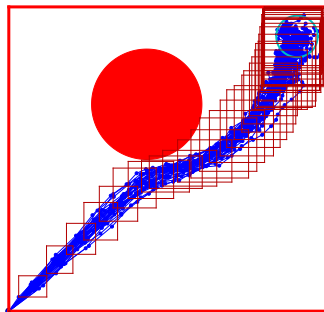
Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

Remarks

- The propagation of the “containers” in the model predictions can be expensive / difficult
- Pessimistic propagations are usually needed → pessimistic over pessimistic
- Policy based on worst-case perspective makes the decisions highly conservative
- Often labelled “Robust” decision making

Trajectories with “containers”



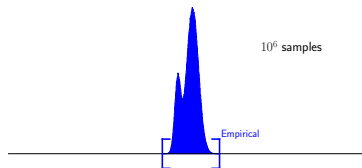
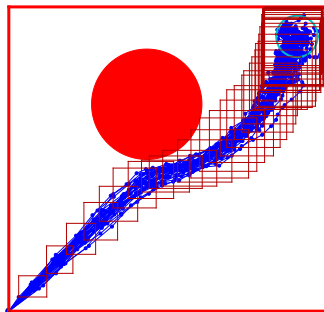
Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

Epistemic problem: build container...

- from data, but it may expand with more data and safety is “asymptotic”, as $\text{data} \rightarrow \infty$

Trajectories with “containers”



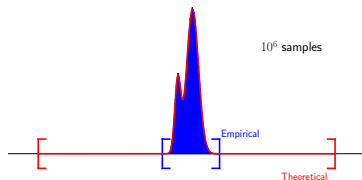
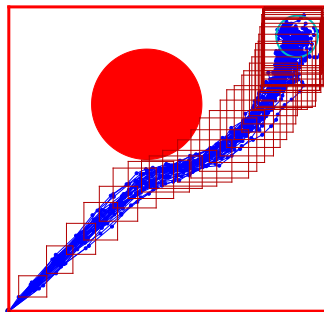
Pessimistic Models for Decision Making

- Model must “contain” the uncertainty
- Trajectories predicted by pessimistic model will “cover” the real world
- “Container” (set) should be simple for computational reasons
- Decision policy wants to be safe w.r.t. the “containers”

Epistemic problem: build container...

- from data, but it may expand with more data and safety is “asymptotic”, as data $\rightarrow \infty$
- “theoretically” (from knowledge), but can be impractically pessimistic

Trajectories with “containers”



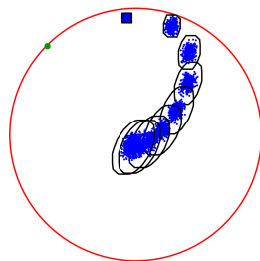
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future



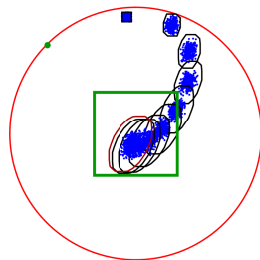
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening



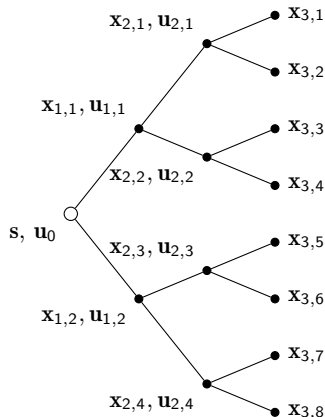
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening



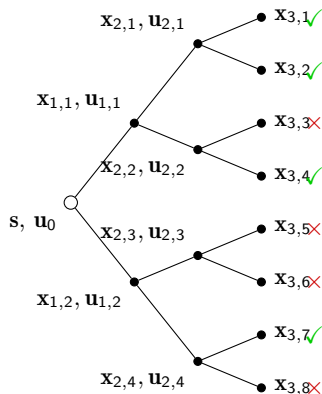
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



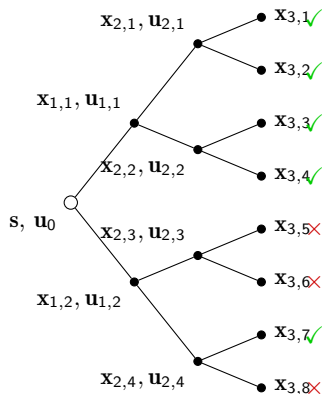
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



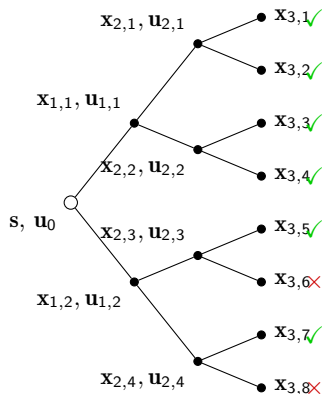
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



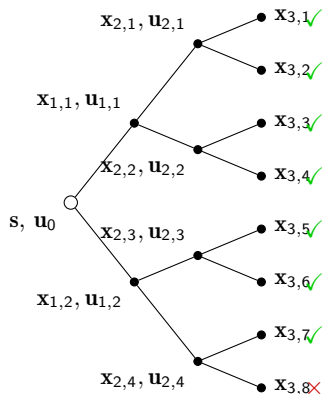
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



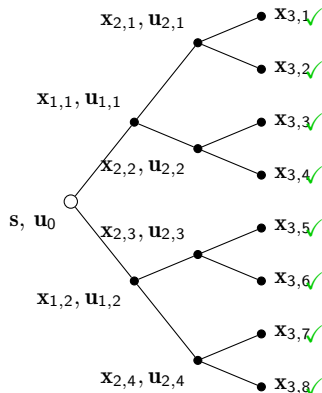
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



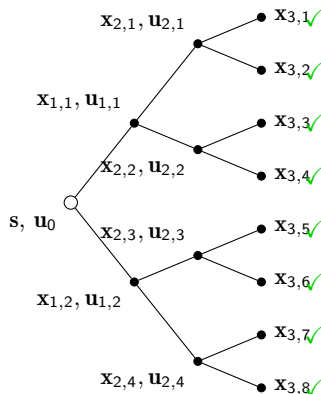
Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.



Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.
- **Sampling:** Monte Carlo for large-scale problems

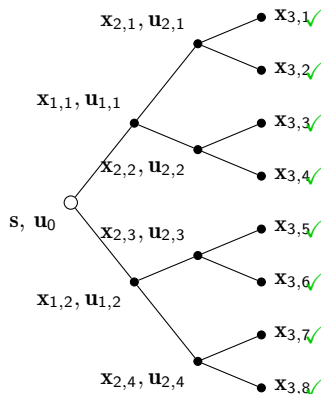


Robust Repeated Planning

- Re-plan at each stage, hedging against worst case uncertainties in the future
- **Robust MPC:** constraint tightening
- **Scenario trees Stochastic programming:** hedge decisions across all branched futures.
- **Sampling:** Monte Carlo for large-scale problems

Remarks:

- In practice: robust optimization = **min-max**: maximize utility over scenarios that avoid worst-case violation.
- Hard to establish guarantees for nonlinear systems
- Scenario explosion.
- Persistent safety (*recursive feasibility*) under re-planning.



Risk-sensitive MDPs

In words

- No notion of Constraints
- Maximize the utility of bad outcomes
- Modify the utility to account for risk

Risk-sensitive MDPs

In words

- No notion of Constraints
- Maximize the utility of bad outcomes
- Modify the utility to account for risk

Formally

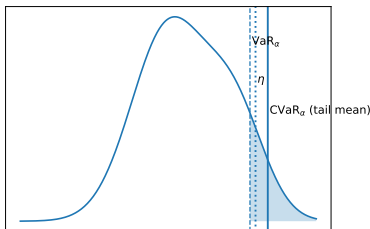
$$\pi^* = \arg \max_{\pi} \rho \left(\sum_{t=0}^{\infty} \gamma^t r_t \right)$$

ρ : VaR, CVaR, Entropic Risk.

Risk-sensitive MDPs

In words

- No notion of Constraints
- Maximize the utility of bad outcomes
- Modify the utility to account for risk



Formally

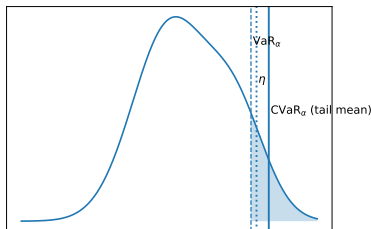
$$\pi^* = \arg \max_{\pi} \rho \left(\sum_{t=0}^{\infty} \gamma^t r_t \right)$$

ρ : VaR, CVaR, Entropic Risk.

Risk-sensitive MDPs

In words

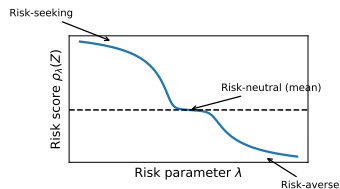
- No notion of Constraints
- Maximize the utility of bad outcomes
- Modify the utility to account for risk



Formally

$$\pi^* = \arg \max_{\pi} \rho \left(\sum_{t=0}^{\infty} \gamma^t r_t \right)$$

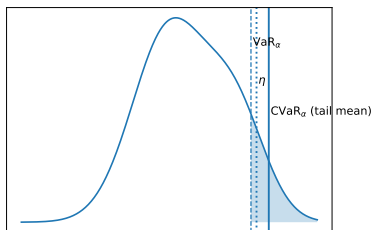
ρ : VaR, CVaR, Entropic Risk.



Risk-sensitive MDPs

In words

- No notion of Constraints
- Maximize the utility of bad outcomes
- Modify the utility to account for risk



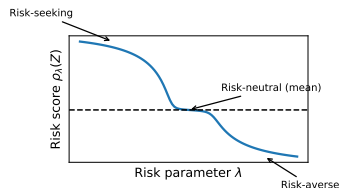
Remarks:

- Views risk through the lens of utility (not undesirable events).
- Risk-sensitive criteria do *not* guarantee a bound on violation.
- Cares only about the average severity of bad outcomes in the tail and not about the rest.

Formally

$$\pi^* = \arg \max_{\pi} \rho \left(\sum_{t=0}^{\infty} \gamma^t r_t \right)$$

ρ : VaR, CVaR, Entropic Risk.

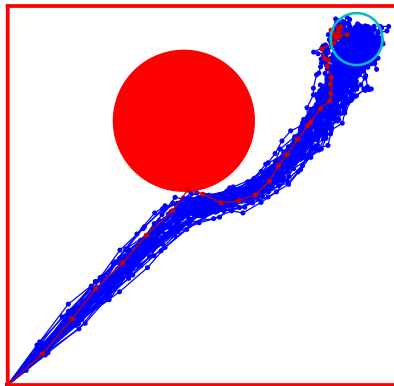


MDPs with probabilistic safety

In words

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Probability of no violation $\geq c$



MDPs with probabilistic safety

In words

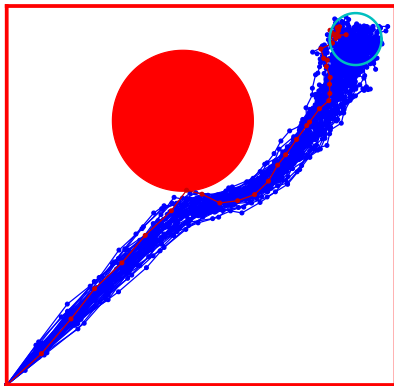
$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Probability of no violation $\geq c$

Formally

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, a_k) \right]$$

s.t. $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}] \geq c$



MDPs with probabilistic safety

In words

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{\text{time}} \text{Utility} \right]$$

s.t. Probability of no violation $\geq c$

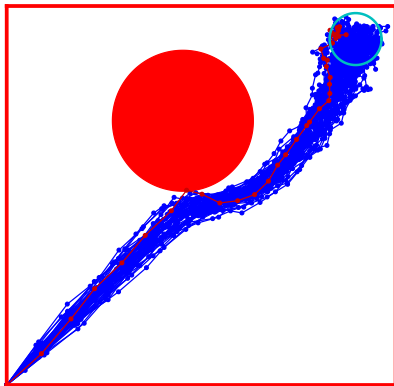
Formally

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, a_k) \right]$$

s.t. $P[s_0, \dots, \infty \in \mathbb{S}] \geq c$

Remarks:

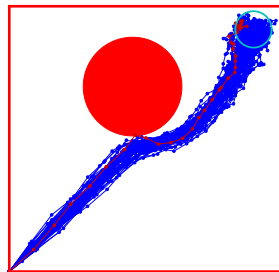
- If we can tolerate $c > 0$ (small) it can make a big gain in performance
- Aligned with industrial / practical standards on “large series”
- Problem needs a “termination” (time or goal reached)
- Building decisions can be difficult from a computational point of view



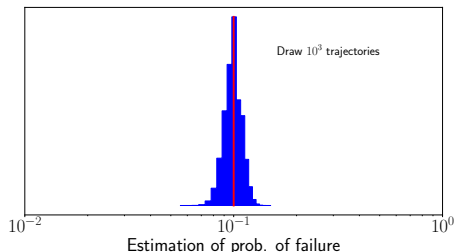
Solving Problems with Probabilistic Safety

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, \mathbf{a}_k) \right]$$

s.t. $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}] \geq c$



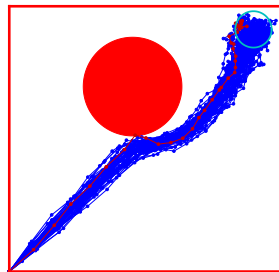
- Classical techniques (e.g. DP and RL) apply but they become very data intensive
- Difficulties: estimate expected values from data (real or not) when $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}]$ is close to 1.



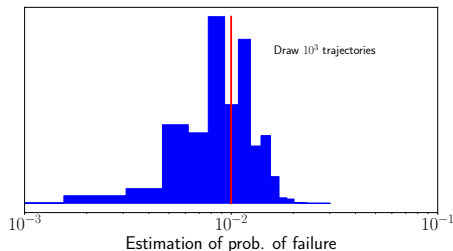
Solving Problems with Probabilistic Safety

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, \mathbf{a}_k) \right]$$

s.t. $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}] \geq c$



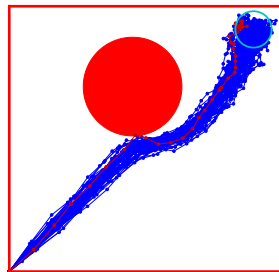
- Classical techniques (e.g. DP and RL) apply but they become very data intensive
- Difficulties: estimate expected values from data (real or not) when $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}]$ is close to 1.



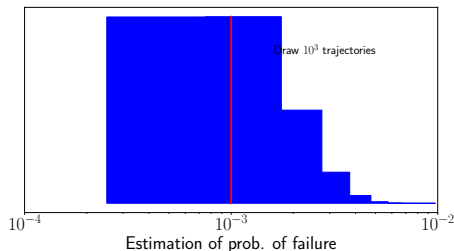
Solving Problems with Probabilistic Safety

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, \mathbf{a}_k) \right]$$

s.t. $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}] \geq c$



- Classical techniques (e.g. DP and RL) apply but they become very data intensive
- Difficulties: estimate expected values from data (real or not) when $\mathbb{P}[s_0, \dots, \infty \in \mathbb{S}]$ is close to 1.



Outline

- 1 Some Basics of Safe Decision Making
- 2 Safe Decisions from Data & AI
- 3 Epistemic Uncertainty and Safe Decisions

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs



Train policy π_θ via evaluations of $\nabla_\theta J(\pi_\theta)$ and

$$J(\pi_\theta) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy



Train policy π_θ via evaluations of $\nabla_\theta J(\pi_\theta)$ and

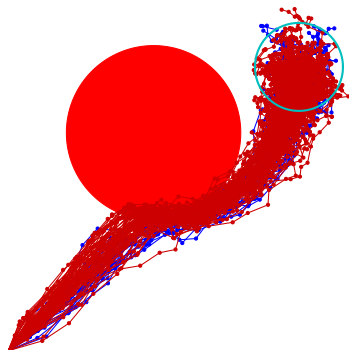
$$J(\pi_\theta) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

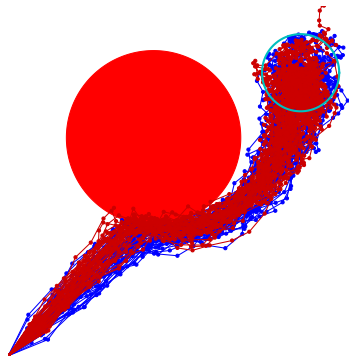
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

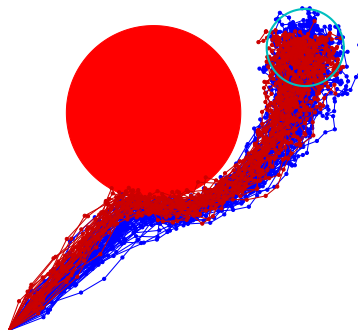
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

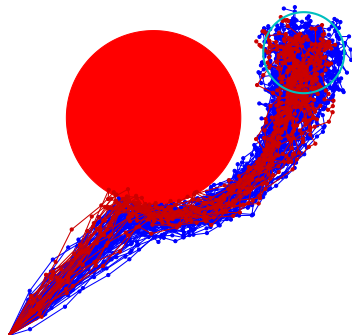
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

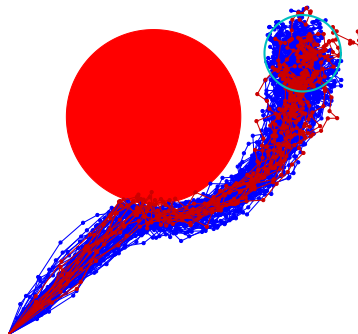
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

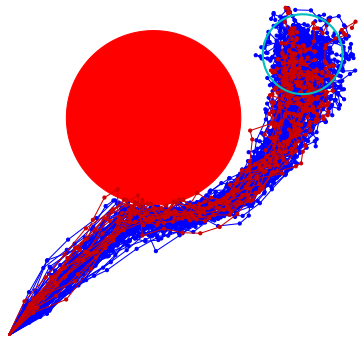
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

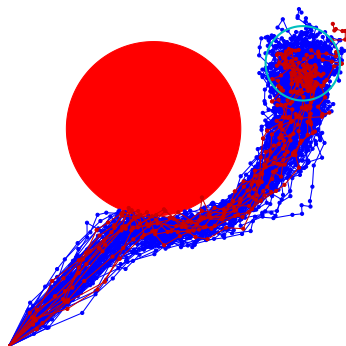
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

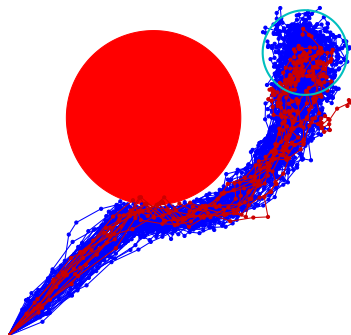
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

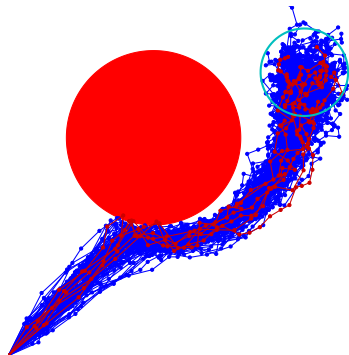
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

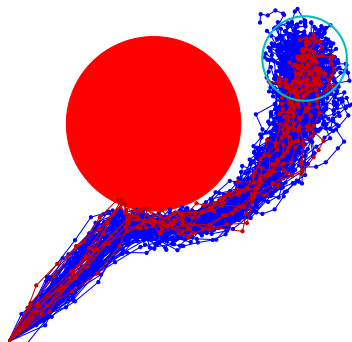
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

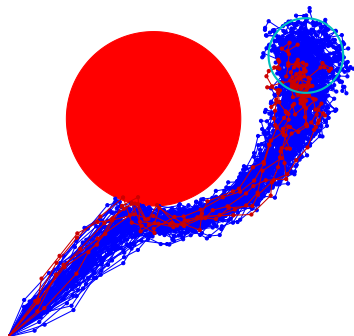
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

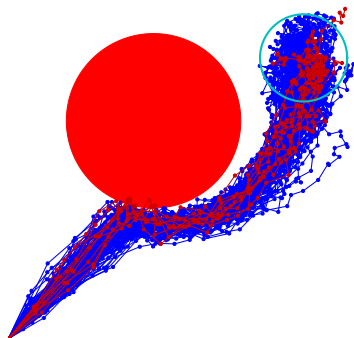
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

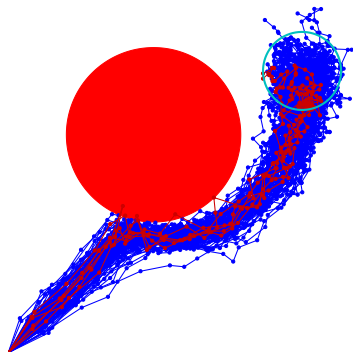
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

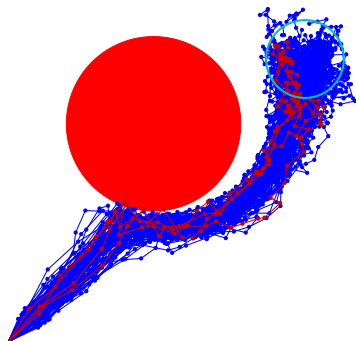
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

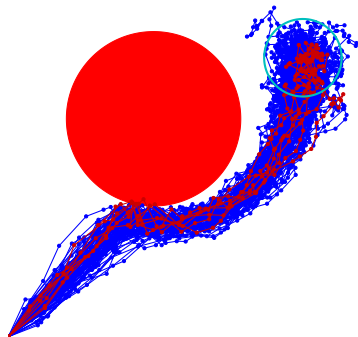
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

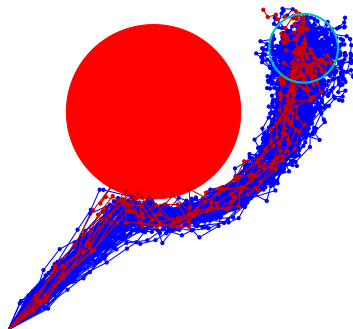
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

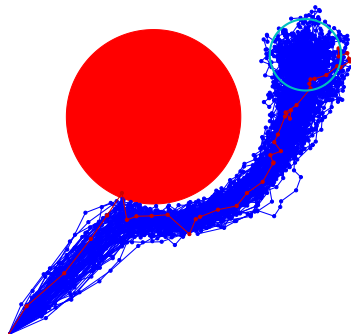
$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

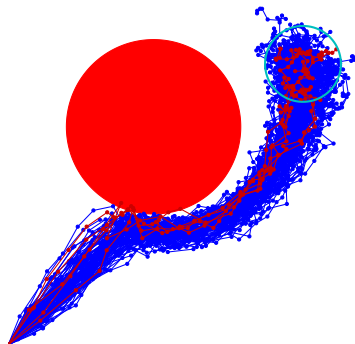
Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare

Difficulties:

- Noisy gradient when violations become rare



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

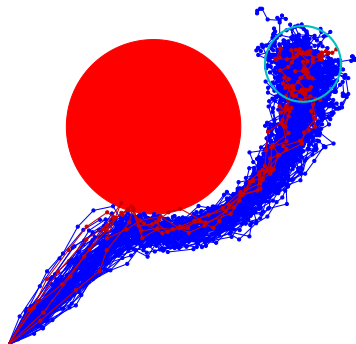
Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare

Difficulties:

- Noisy gradient when violations become rare
- “Hard” safety is never reached (asymptotic)



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

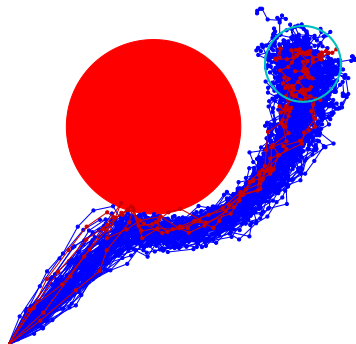
Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare

Difficulties:

- Noisy gradient when violations become rare
- “Hard” safety is never reached (asymptotic)
- The higher (probabilistic) safety we want the more data we need



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

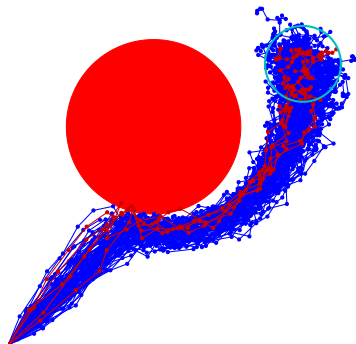
Reinforcement Learning & Safety

Key idea: Add a strong penalty to the utility when a violation occurs

- May start with a “fully unsafe” policy
- Following gradient $\nabla_{\theta} J(\pi_{\theta})$ reduces violations
- But safety improvement “stalls” as violations become more rare

Difficulties:

- Noisy gradient when violations become rare
- “Hard” safety is never reached (asymptotic)
- The higher (probabilistic) safety we want the more data we need



Train policy π_{θ} via evaluations of $\nabla_{\theta} J(\pi_{\theta})$ and

$$J(\pi_{\theta}) = \mathbb{E} \left[\sum_{\text{time}} \left(\text{Utility} - \begin{cases} 0 & \text{if Constraint ok} \\ p & \text{if Constraint not ok} \end{cases} \right) \right]$$

for p large

Unrealistic in the real world, it's a simulation-based process

Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$

Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

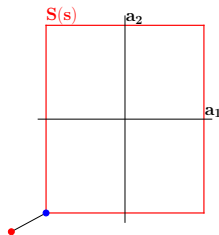
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



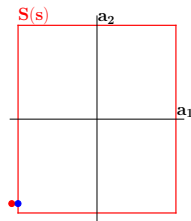
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



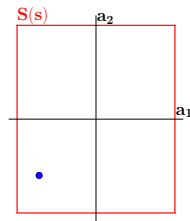
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



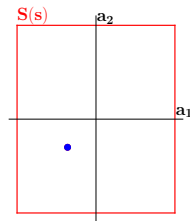
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



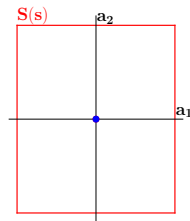
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



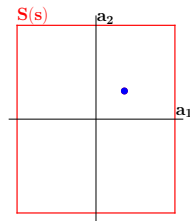
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



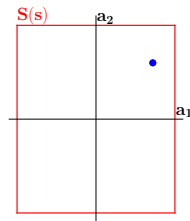
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



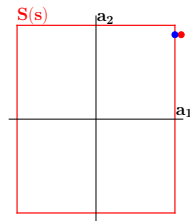
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



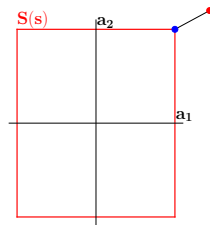
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



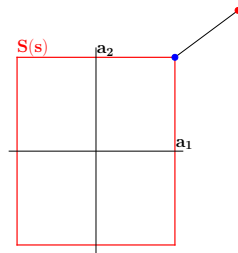
Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$



Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

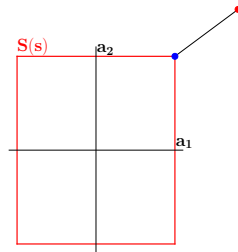
that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$

Remark: “oracle” can be very hard to build, or not...



Not losing the king



Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$

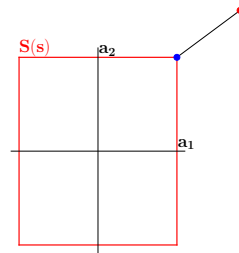
Remark: “oracle” can be very hard to build, or not...



Not losing the king



Able to stop within the visible distance



Safety Filters & Reinforcement Learning

- For any state s , an “oracle” tells you which actions will not jeopardize safety in the long run $\rightarrow$ “Safe set” $S(s)$
- We can learn an “unsafe” policy

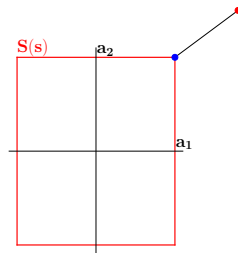
$$\pi^* = \arg \max_{\pi} E \left[\sum_{\text{time}} \text{Utility} \right]$$

that focuses only on utility

- For state s , take the decisions using: Action = $\text{Projection}_{S(s)} [\pi^*(s)]$

Remarks:

- Often close to practice, e.g., flight envelope protection in modern airplane, semi-autonomous driving in cars, etc
- Oracle $\Rightarrow$ knowledge-based & conservative!



Control Barrier Functions (CBFs)

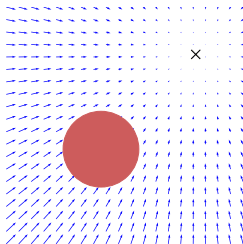
- CBFs provide a formal, model-based way to build the “oracle” safe set

Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$

Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$

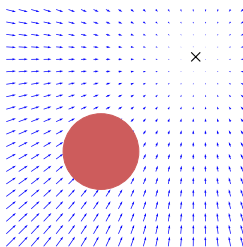


Vector field without CBF filter.

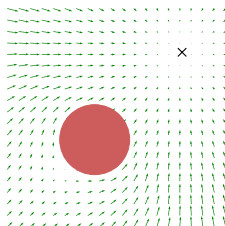
Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$
- CBF modifies the nominal decision by solving a QP

$$u^*(x) = \arg \min_u \frac{1}{2} \|u - u_{\text{des}}(x)\|_2^2 \quad \text{s.t. derivative of the } h(x) \text{ along the vector field} \geq 0$$



Vector field without CBF filter.

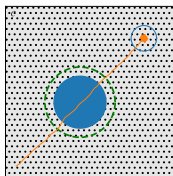


vector field with CBF filter.

Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$
- CBF modifies the nominal decision by solving a QP

$$u^*(x) = \arg \min_u \frac{1}{2} \|u - u_{\text{des}}(x)\|_2^2 \quad \text{s.t.} \quad \text{derivative of the } h(x) \text{ along the vector field} \geq 0$$

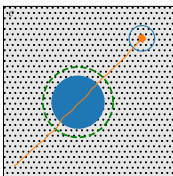


Unsafe trajectory: violates $h(x) \geq 0$.

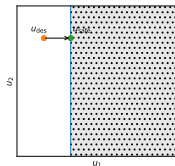
Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$
- CBF modifies the nominal decision by solving a QP

$$u^*(x) = \arg \min_u \frac{1}{2} \|u - u_{\text{des}}(x)\|_2^2 \quad \text{s.t.} \quad \text{derivative of the } h(x) \text{ along the vector field} \geq 0$$



Unsafe trajectory: violates $h(x) \geq 0$.

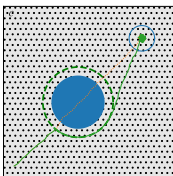


CBF filter: project u_{des} to feasible u_{safe} .

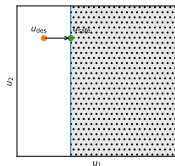
Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$
- CBF modifies the nominal decision by solving a QP

$$u^*(x) = \arg \min_u \frac{1}{2} \|u - u_{\text{des}}(x)\|_2^2 \quad \text{s.t.} \quad \text{derivative of the } h(x) \text{ along the vector field} \geq 0$$



CBF-filtered trajectory.



CBF filter: project u_{des} to feasible u_{safe} .

Control Barrier Functions (CBFs)

- CBFs provide a formal, model-based way to build the “oracle” safe set
- Use a barrier function $h(x)$ so that staying safe means staying in set $\mathcal{C} = \{x : h(x) \geq 0\}$
- CBF modifies the nominal decision by solving a QP

$$u^*(x) = \arg \min_u \frac{1}{2} \|u - u_{\text{des}}(x)\|_2^2 \quad \text{s.t. derivative of the } h(x) \text{ along the vector field } \geq 0$$

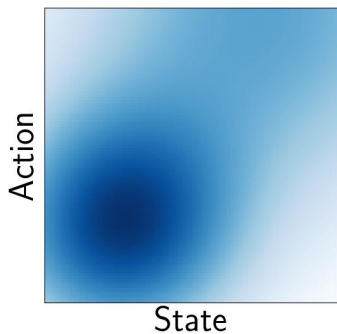
Remarks:

- The barrier function and safe set $\mathcal{C}$ are typically constructed from domain knowledge (physics, rules, safety envelopes).
- Requires a (possibly simplified, conservative) system model: robust approach.

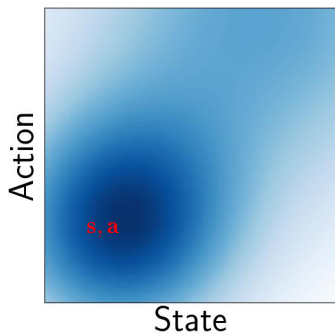
Outline

- 1 Some Basics of Safe Decision Making
- 2 Safe Decisions from Data & AI
- 3 Epistemic Uncertainty and Safe Decisions

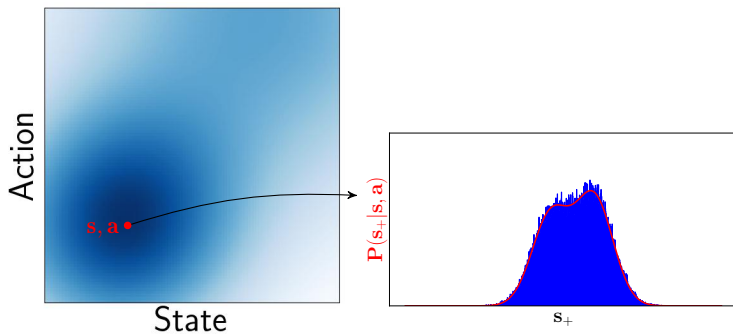
Data & Epistemic Uncertainty



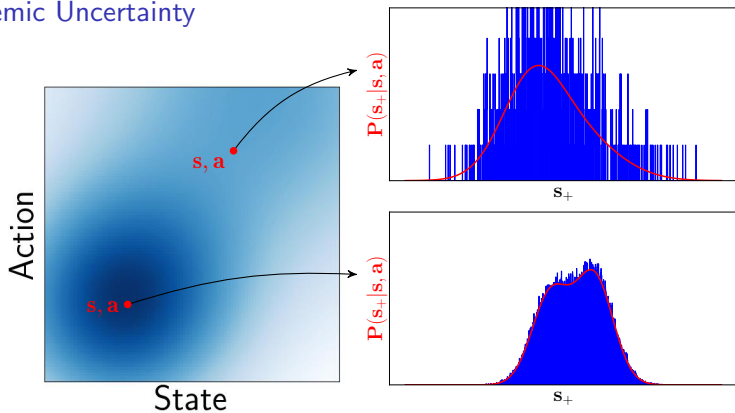
Data & Epistemic Uncertainty



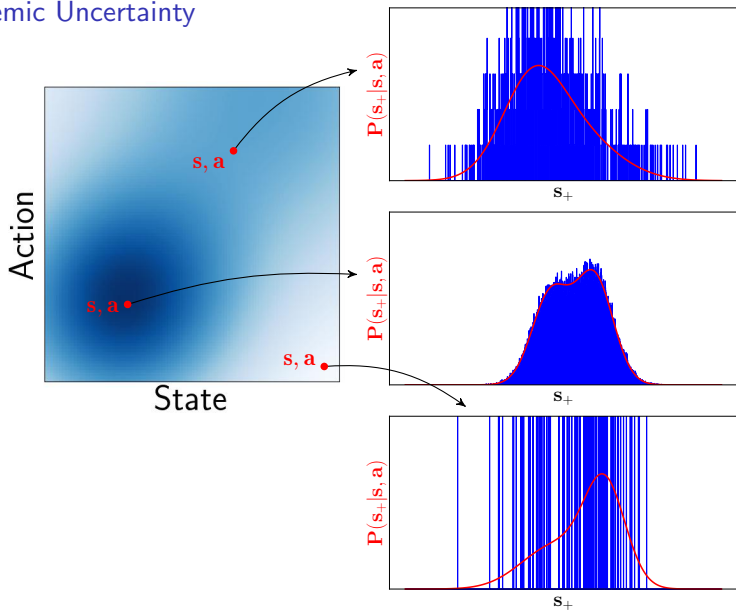
Data & Epistemic Uncertainty



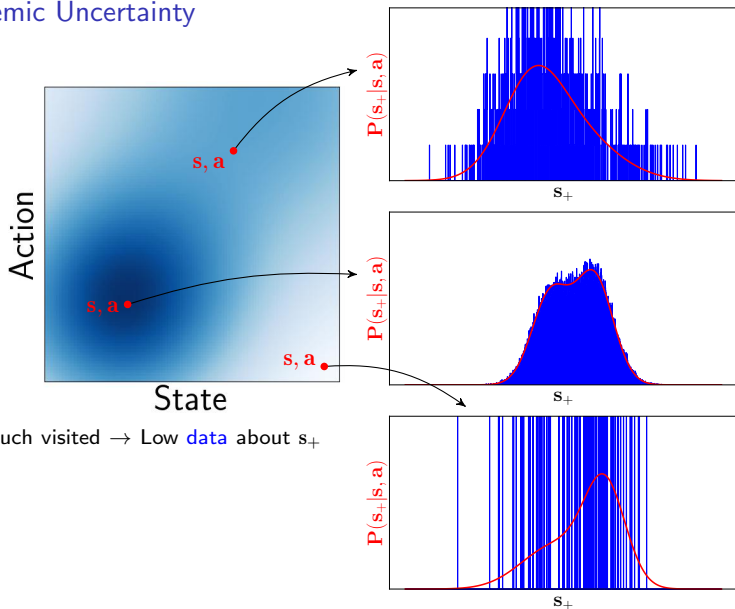
Data & Epistemic Uncertainty



Data & Epistemic Uncertainty



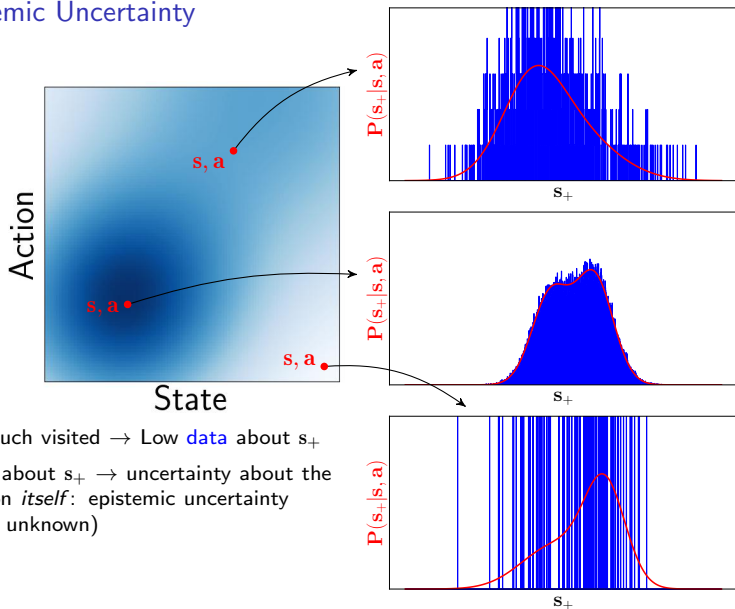
Data & Epistemic Uncertainty



Remarks

- s, a not much visited $\rightarrow$ Low data about s_+

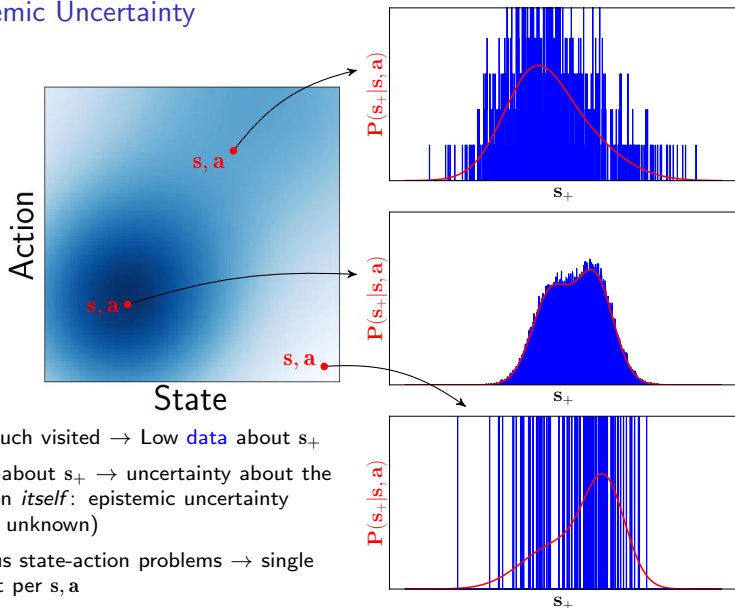
Data & Epistemic Uncertainty



Remarks

- **s, a** not much visited → Low **data** about s_+
- Low data about s_+ → uncertainty about the distribution *itself*: epistemic uncertainty (unknown unknown)

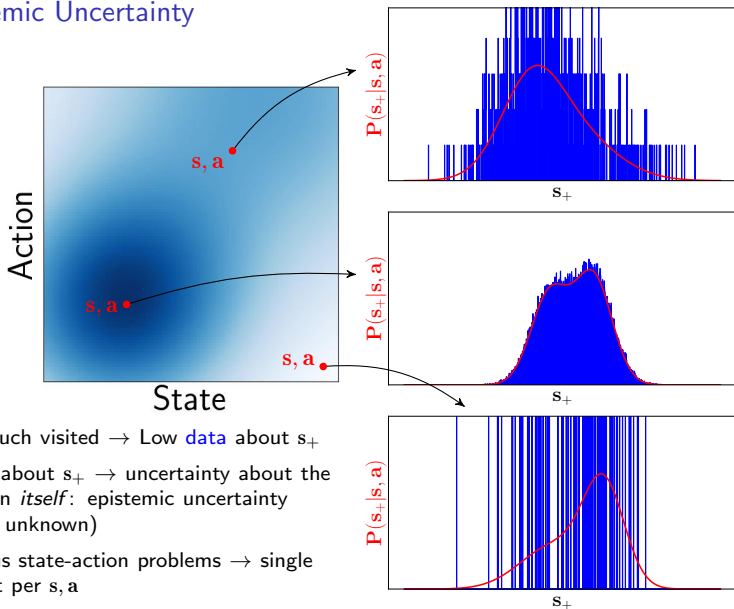
Data & Epistemic Uncertainty



Remarks

- s, a not much visited $\rightarrow$ Low data about s_+
- Low data about $s_+ \rightarrow$ uncertainty about the distribution *itself*: epistemic uncertainty (unknown unknown)
- Continuous state-action problems $\rightarrow$ single data point per s, a

Data & Epistemic Uncertainty



Remarks

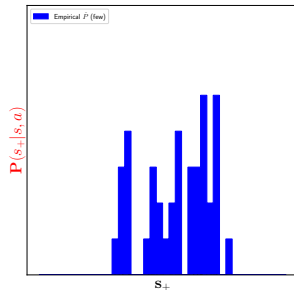
- s, a not much visited $\rightarrow$ Low data about s_+
- Low data about s_+ $\rightarrow$ uncertainty about the distribution *itself*: epistemic uncertainty (unknown unknown)
- Continuous state-action problems $\rightarrow$ single data point per s, a
- Rely on regularity across s, a !!

Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available

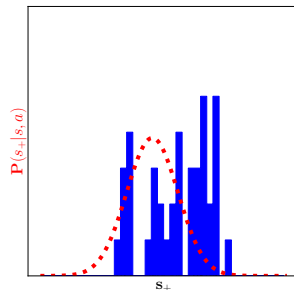
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available



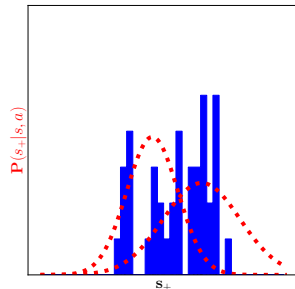
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available



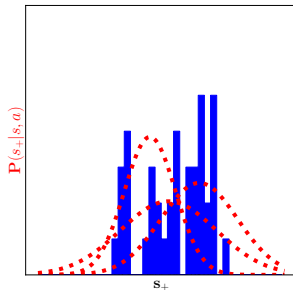
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



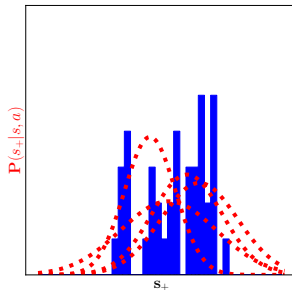
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



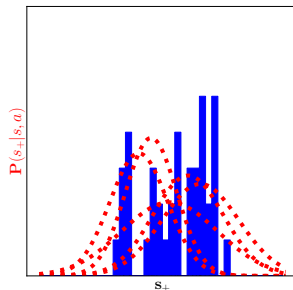
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



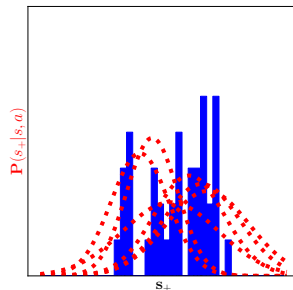
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



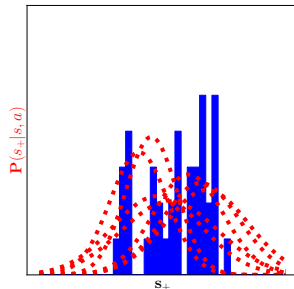
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



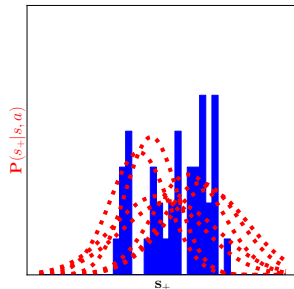
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions



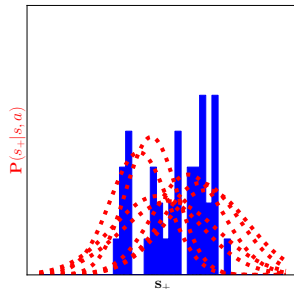
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions
- Knowledge embedding: encode physics/rules as priors or constraints on models



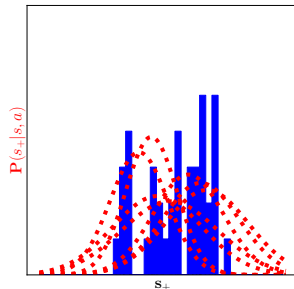
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions
- Knowledge embedding: encode physics/rules as priors or constraints on models
- Bayesian view: Data updates the prior belief about the model providing a posterior



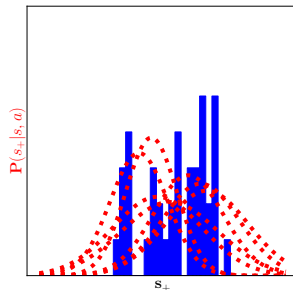
Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions
- Knowledge embedding: encode physics/rules as priors or constraints on models
- Bayesian view: Data updates the prior belief about the model providing a posterior
- Building $\mathcal{P}$: ensembles, parametric, non-parametric



Distributional View on MDPs

- True dynamics are unknown; only samples or estimates available
- Instead of a single distribution, a family of distributions
- Knowledge embedding: encode physics/rules as priors or constraints on models
- Bayesian view: Data updates the prior belief about the model providing a posterior
- Building $\mathcal{P}$: ensembles, parametric, non-parametric



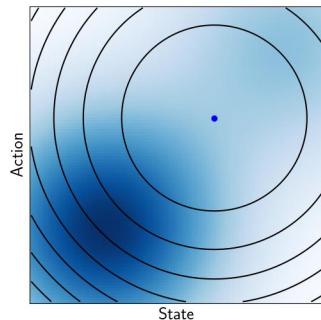
Remarks:

- Accounts for **epistemic uncertainty** from limited data; connects to probabilistic safety via confidence levels.
- Distributionally Robust Optimization for worst-case robustness against model misspecification (more conservative).
- Expectation over distribution of distributions: some form of discretization. Bayesian marginalization or Scenario discretization

Out-Of-Distribution (OOD) “Guarding” Methods

Utility + model may “push” a problem to get “Out-Of-Distribution”

- Unwarranted extrapolation / generalization
- Optimization often exploits the model “weaknesses”

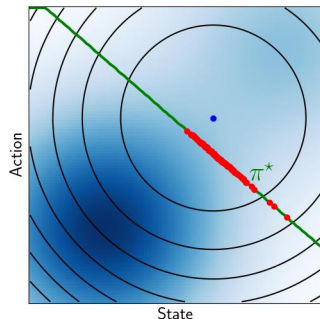


Out-Of-Distribution (OOD) “Guarding” Methods

Utility + model may “push” a problem to get “Out-Of-Distribution”

- Unwarranted extrapolation / generalization
- Optimization often exploits the model “weaknesses”

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \left(\mathbf{s}_k^2 + \mathbf{a}_k^2 \right) \right]$$



Out-Of-Distribution (OOD) “Guarding” Methods

Utility + model may “push” a problem to get “Out-Of-Distribution”

- Unwarranted extrapolation / generalization
- Optimization often exploits the model “weaknesses”

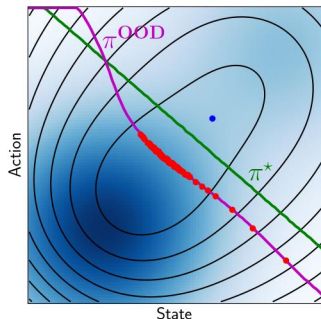
$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \left(s_k^2 + a_k^2 \right) \right]$$

Key idea

- Max utility while penalizing OOD. E.g.

$$\pi^{\text{OOD}} = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \left(s_k^2 + a_k^2 - \omega \cdot \varphi(s_k, a_k) \right) \right]$$

where φ is the data density (e.g. kernel)



Out-Of-Distribution (OOD) “Guarding” Methods

Utility + model may “push” a problem to get “Out-Of-Distribution”

- Unwarranted extrapolation / generalization
- Optimization often exploits the model “weaknesses”

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \left(s_k^2 + a_k^2 \right) \right]$$

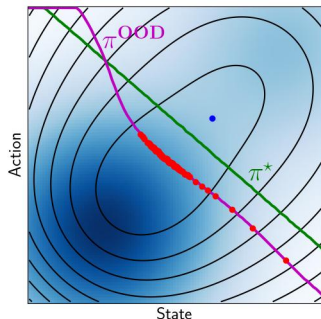
Key idea

- Max utility while penalizing OOD. E.g.

$$\pi^{\text{OOD}} = \arg \max_{\pi} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \left(s_k^2 + a_k^2 - \omega \cdot \varphi(s_k, a_k) \right) \right]$$

where φ is the data density (e.g. kernel)

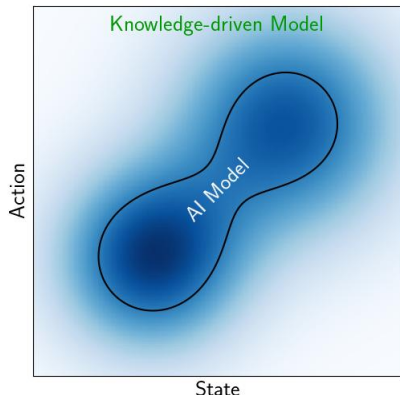
- As we get more data, $\pi^{\text{OOD}} \rightarrow \pi^*$ (where it is useful)



Out-Of-Distribution (OOD) Hybrid Model Methods

Key idea: use AI when we are “within data”, and knowledge when OOD

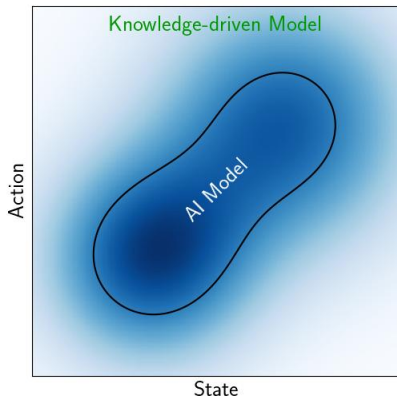
- Transition $AI \leftrightarrow$ Knowledge should be smooth
- Knowledge-driven model can still be data-driven, but with simpler structure than AI model (e.g. linear)
- Tends to outperform AI or Knowledge alone in decision making
- Approach is heuristic, lack of theoretical foundations



Out-Of-Distribution (OOD) Hybrid Model Methods

Key idea: use AI when we are “within data”, and knowledge when OOD

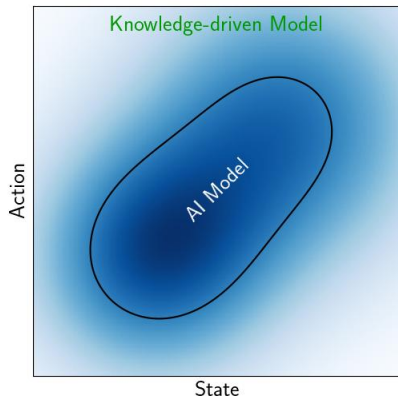
- Transition $AI \leftrightarrow$ Knowledge should be smooth
- Knowledge-driven model can still be data-driven, but with simpler structure than AI model (e.g. linear)
- Tends to outperform AI or Knowledge alone in decision making
- Approach is heuristic, lack of theoretical foundations



Out-Of-Distribution (OOD) Hybrid Model Methods

Key idea: use AI when we are “within data”, and knowledge when OOD

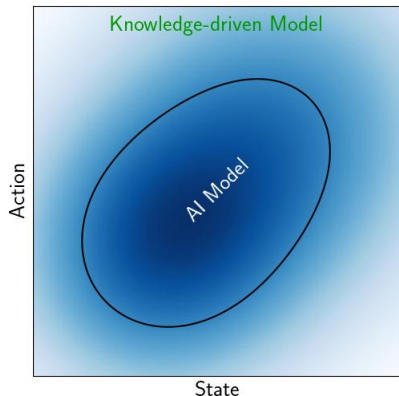
- Transition $AI \leftrightarrow$ Knowledge should be smooth
- Knowledge-driven model can still be data-driven, but with simpler structure than AI model (e.g. linear)
- Tends to outperform AI or Knowledge alone in decision making
- Approach is heuristic, lack of theoretical foundations



Out-Of-Distribution (OOD) Hybrid Model Methods

Key idea: use AI when we are “within data”, and knowledge when OOD

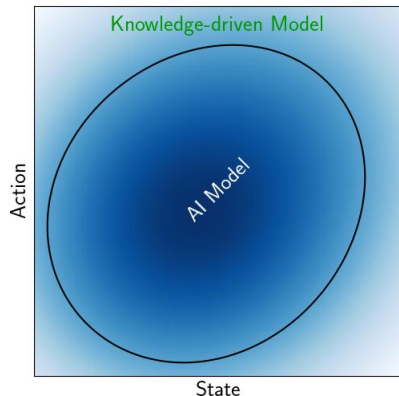
- Transition $AI \leftrightarrow$ Knowledge should be smooth
- Knowledge-driven model can still be data-driven, but with simpler structure than AI model (e.g. linear)
- Tends to outperform AI or Knowledge alone in decision making
- Approach is heuristic, lack of theoretical foundations



Out-Of-Distribution (OOD) Hybrid Model Methods

Key idea: use AI when we are “within data”, and knowledge when OOD

- Transition $AI \leftrightarrow Knowledge$ should be smooth
- Knowledge-driven model can still be data-driven, but with simpler structure than AI model (e.g. linear)
- Tends to outperform AI or Knowledge alone in decision making
- Approach is heuristic, lack of theoretical foundations



What does it say about AID?

We have discussed on the key concepts of safe decision making

What does it say about AID?

We have discussed on the key concepts of safe decision making

RA6 (Extrapolation & Generalization)

- Data-based models have epistemic uncertainty
- Decision making will often push the models outside their validity (unwarranted extrapolation)
- How to manage this problem better is a key RA in AID

What does it say about AID?

We have discussed on the key concepts of safe decision making

RA6 (Extrapolation & Generalization)

- Data-based models have epistemic uncertainty
- Decision making will often push the models outside their validity (unwarranted extrapolation)
- How to manage this problem better is a key RA in AID

RA4 (Knowledge embedding)

- Knowledge is often key to manage epistemic uncertainty
- How to embed knowledge in AI for Decision to manage extrapolation better is a key RQ in AID

What does it say about AID?

We have discussed on the key concepts of safe decision making

RA5 (Training models for decision)

- Best model fit $\neq$ best model for decision
- The problem of generating relevant scenarios (or probability mass) for safe decision making is strongly connected (same theory)
- How to train models to give us predictions conducive to safety is a key RQ in AID
- Especially open for probabilistic safety guarantees

What does it say about AID?

We have discussed on the key concepts of safe decision making

RA5 (Training models for decision)

- Best model fit $\neq$ best model for decision
- The problem of generating relevant scenarios (or probability mass) for safe decision making is strongly connected (same theory)
- How to train models to give us predictions conducive to safety is a key RQ in AID
- Especially open for probabilistic safety guarantees

RA7 (AI and formal decision tools)

- Formal decision tools offer a pathway to ensure safe decisions
- E.g. robust repeated planning
- How to combine these tools in a good way with AI is an active research topic. Especially how to make it work in practice.